Lab 4 (CUT, FAIL)

What is cut (!) in Prolog?

The **cut** operator **!** is used to control backtracking in Prolog.

- It commits to the choices made so far in the current rule.
- When Prolog encounters a cut, it will not try other alternative rules or branches before the cut even if the later part fails.

Syntax:

```
predicate :- condition1, !, condition2.
```

- If condition1 is true and ! is reached, Prolog commits to this rule.
- If condition2 fails, it won't go back to try other definitions of predicate.

Example without cut:

```
max(X, Y, X) :- X >= Y.
max(X, Y, Y) :- X < Y.
```

Query:

```
?- max(3, 3, M).
M = 3 ;
M = 3.
```

You get two answers because both clauses succeed when X = Y

With cut:

```
max(X, Y, X) :- X >= Y, !.
max(X, Y, Y).
```

Query:

```
?- max(3, 3, M).
M = 3.
```

Now you get only one answer — the second rule is not tried because of the cut.

Example Without Cut:

```
age_group(Age, child) :- Age < 13.
age_group(Age, teen) :- Age >= 13, Age < 20.
age_group(Age, adult) :- Age >= 20.
```

Query:

```
?- age_group(15, Group)
```

Step-by-Step Execution:

1. **Prolog checks the first rule:**

```
age_group(15, child)
→ 15 < 13 → false  → Try next rule
```

2. **Second rule:**

```
age_group(15, teen)
→ 15 >= 13 → true
→ 15 < 20 → true
```

Rule matches → Group = teen

3. **Since there is no cut (!), Prolog leaves a choice point and may check the third rule if you press ; (semicolon).**
4. **Third rule:**

```
age_group(15, adult)
→ 15 >= 20 → false
```

Final Output:

```
Group = teen ;
false.
```

Example With Cut to Prune Choices:

```
age_group(Age, child) :- Age < 13, !.
age_group(Age, teen) :- Age < 20, !.
age_group(_, adult).
```

Explanation:

- ✓ If Age < 13 is true, child is selected and no other rules are tried — thanks to !
- ✓ If not, check Age < 20 — if true, teen is selected, and cut stops further search.
- ✓ Only if both fail, it chooses adult**.**

Sample Queries and Behavior:

```
?- age_group(10, G).
G = child.    % Only child is matched — no backtracking

?- age_group(15, G).
G = teen.     % Matches teen and cuts further

?- age_group(30, G).
G = adult.    % Both conditions fail, matches default
```

Why "prune choices"?

- ✓ Without the cut:
  Prolog may leave choice points (e.g., teen and adult) even after matching teen.
- ✓ With the cut:
  Once a condition is matched, no unnecessary rule is evaluated.
  This improves performance and clarity.

Types of Cuts

Green cut: Used to improve efficiency, does not change logic (safe).
Red cut: Affects logic & behavior; removes valid answers (dangerous if misused).

What is a Green Cut?

A green cut is used only to prevent unnecessary backtracking — it does not alter which solutions Prolog would find without it.

Sample Program: Classify Age

```
% Green cut version
age_group(Age, child) :- Age < 13, !.
age_group(Age, teen)  :- Age < 20, !.
age_group(_, adult).
```

Without the cut:

Prolog tries each rule even after a match (leaving choice points).

With the green cut:

Once the correct rule fires, Prolog won't try the rest — but results remain the same.

Example Queries:

```
?- age_group(10, Group).
Group = child.

?- age_group(15, Group).
Group = teen.

?- age_group(30, Group).
Group = adult.
```

What Is a Red Cut?

A red cut is used in a way that affects the logical meaning of the program.
It forces a commitment to a rule, even if it may not be logically the best or correct one.

Red Cut Example: Max of Two Numbers

```
max(X, Y, X) :- X >= Y, !.
max(_, Y, Y).
```

Explanation:

- ✓ If X >= Y, Prolog returns X and cuts all other options
- ✓ Else, it returns Y
- ✓ Red cut is used to prevent the second rule from being tried, even if the condition in the first rule is wrong due to a bug or bad input.

Buggy Output Example:

```
?- max(5, five, M).
```

This may fail completely due to type mismatch in comparison — but Prolog might not backtrack!

Because:

- ✓ First rule fails or behaves oddly
- ✓ Cut still prevents trying the second rule

Safer Red Cut Usage

```
max(X, Y, X) :- number(X), number(Y), X >= Y, !.
max(_, Y, Y).
```

What is fail in Prolog?

The **fail** predicate always fails deliberately. It's used:

- ✓ To force failure
- ✓ Often in combination with **!** to prune choices or implement negation

Syntax:

```
predicate :- condition, !, fail.
```

This causes the rule to always fail, but cut prevents trying other rules.

Example with cut and fail (negation):

```
not(P) :- P, !, fail.
not(_).
```

EXAMPLE: A goal that succeeds

Let's define a simple fact:

```
sun(shines).
```

Now run the query:

```
?- not(sun(shines)).
```

Step-by-step:

First clause: `not(P) :- P, !, fail.`
- ✓ **Try** `P = sun(shines)`
- ✓ `sun(shines)` **succeeds**

`! (cut)` is reached → Prolog commits to this clause
`fail` → Prolog fails the goal intentionally
It does not try the second clause due to the cut

Final result:

```
false.
```

EXAMPLE: A goal that fails

```
?- not(sun(rises)).
```

Assume there's no fact `sun(rises).`

Step-by-step:

First clause: `not(P) :- P, !, fail.`
- ✓ **Try** `P = sun(rises)`
- ✓ `sun(rises)` **fails**

So, this clause fails entirely
Prolog moves to the second clause: `not(_).`
That clause succeeds

What happens if you remove fail from the classic custom negation definition in Prolog:

Suppose you write:

```
not(P) :- P, !.
not(_).
```

Now the clause becomes:

```
If P is true, cut, then... do nothing and succeed!
```

Example: With `fact bird(sparrow)`

Query:

```
?- not(bird(sparrow)).
```

Execution (without `fail`**):**

Try `bird(sparrow)` → true
Hit `! (cut)` → **commit to this rule**
No failure → succeeds!

Output:

```
true.
```

Which is wrong for negation logic — it says the bird is not a sparrow, even though it is.

Example: Filtering with cut and fail

```
even(X) :- 0 is X mod 2.
odd(X) :- even(X), !, fail.
odd(_).
```

Query:

```
?- odd(3).    % Not even → fail cut → fallback to odd(_)
true.
?- odd(4).    % Even → cut → fail → no fallback
false.
```

Assignment Task: Filter out even numbers from a list using cut and fail

Write a Prolog program that:

- Takes a list of integers
- Removes all even numbers
- Keeps only odd numbers
- Uses cut (!) and fail for control flow

Assignment Title: "Implementing Decision-Based Access Control Using Cut and Fail in Prolog"

Write a Prolog program that decides user access levels (admin, manager, guest, unknown) based on user roles. Use cut and fail to:

- Ensure only one access rule applies
- Block fallback access levels
- Prevent redundant matching

Requirement

You must write a Prolog program that:

1. Defines different user roles (e.g., admin, manager, guest, etc.)
2. Assigns access levels based on roles:

- admin → full access
- manager → limited access
- guest → read-only access

3. Uses cut (!) to prevent Prolog from checking further rules once a match is found
4. Uses fail to explicitly reject unknown roles
5. Prevents fallback access from being applied incorrectly

Example Behavior:
Query 1:

```
?- access(admin, Level).
Level = full_access.
```

Query 2:

```
?- access(manager, Level).
Level = limited_access.
```

Query 3:

```
?- access(foo, Level).
false.  % Unknown role, rejected
```